Studio Nx

P1-12 문의 API — 함수 설명 합본 (백엔드 50~55 · JS 28)

2026-10-08

목차

1. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/50_inquiry_schema_request.md
2. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/51_inquiry_service_rate_limit.md
3. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/52_inquiry_service_create_inquiry.md
4. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/53_inquiries_route_create.md
5. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/54_retention_purge_old_inquiries.md
6. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/55_retention_main.md
7. ai-notes/javascript/P1-12 - 문의 폼 JS/28_home_initContactForm.md

Backend(FastAPI/Python) 50 — `InquiryCreateRequest` (문의 요청 스키마)

항목	내용
파일	`studionx/backend/app/schemas/inquiry.py`
작업	P1-12 문의 API — 요청 본문 검사 · PRD 01 FR-112
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	49 `purge_withdrawn_accounts()` · 51 `check_rate_limit()`

1. 하는 일

홈 화면 **Contact** 폼이 보내는 JSON `{ email, title, content, privacy_agreed }` 의 **모양**을 검사한다. 틀리면 서비스까지 가지 않고 422.

보낸 값	결과
`" you@example.com "`	통과 → `"you@example.com"` (앞뒤 공백 제거)
이메일 `abc` · `kim@naver` · `a@b@c.com` · 빈 글자	422 · `details.email`
제목 `" Bug report "`	통과 → `"Bug report"`
제목 · 내용이 빈 글자 · 공백만 · 줄바꿈만	422 · 그 칸
내용 `"\n line 1\nline 2 \n"`	통과 → `"line 1\nline 2"` — **가운데 줄바꿈은 그대로**
제목 201자 · 내용 2001자	422 (앞뒤 공백은 길이에 안 셈)
`privacy_agreed` 없음	422 — `false` 는 여기서 통과, 서비스(52)가 422 로 막음

2. 언제 쓰나

쓰는 곳	
`POST /api/v1/inquiries` (53)	라우터 매개변수 `data: InquiryCreateRequest` — 틀리면 함수가 불리지도 않고 422 (P1-03 03 처리기)
화면	home.html Contact 폼 (`initContactForm`, 이 작업이 끝나면 연결). 화면도 `type="email"` · `required` · `maxlength` 로 먼저 막고, FIX-01 19 가 말풍선 대신 알림창으로 보여 줌

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from typing import Annotated

from pydantic import BaseModel, EmailStr, Field, StringConstraints
```

`Annotated` · `EmailStr` · `StringConstraints` 를 이번에 추가했다.

상수 · 타입

```
INQUIRY_TITLE_MAX = 200  # 화면 input maxlength 와 같음
INQUIRY_CONTENT_MAX = 2000  # 화면 textarea maxlength 와 같음

# 제목 · 내용 — 앞뒤 공백을 먼저 지우고 길이를 잼 → 공백만 친 칸은 422
# (닉네임 Nickname 과 같은 방법)
# 내용 가운데의 줄바꿈은 그대로 둠
InquiryTitle = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=INQUIRY_TITLE_MAX)
]
InquiryContent = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=INQUIRY_CONTENT_MAX)
]
```

스키마

```
class InquiryCreateRequest(BaseModel):
    email: EmailStr = Field(max_length=255)  # 형식 검사 · 앞뒤 공백 제거 (email-validator)
    title: InquiryTitle
    content: InquiryContent
    privacy_agreed: bool
```

문법 풀이

import 문법	뜻
`from typing import Annotated`	타입에 **검사 규칙을 붙인** 새 타입 이름을 만들 때
`from pydantic import EmailStr, StringConstraints`	이메일 형식 타입 (P1-04 와 같음, `email-validator` 패키지) · 글자 규칙 묶음

식	뜻
`Annotated[str, StringConstraints(…)]`	"글자인데, 이 규칙을 지켜야 함" — 변수처럼 이름(`InquiryTitle`)을 붙여 여러 곳에 씀
`strip_whitespace=True`	앞뒤 공백 · 줄바꿈 · 탭을 **먼저** 지움 → 그다음 `min_length` · `max_length` 검사
`EmailStr = Field(max_length=255)`	이메일 형식 + 길이 (DB 칸 255자)
`privacy_agreed: bool`	반드시 있어야 함 (기본값 없음). `true` 인지는 서비스가 봄

4. 흐름

요청 JSON → pydantic 이 칸마다 검사 (이메일 형식 · 공백 제거 · 길이 · 동의 칸 있음) → 통과하면 깨끗한 값이 든 객체로 라우터에 전달, 하나라도 틀리면 422 + `details` 에 틀린 칸.

5. 왜 이렇게

선택	이유
이메일 형식 검사 = `EmailStr`	템플릿 요구 "P1-04 SignupRequest 와 같은 방법". 답장할 주소이므로 형식이 틀리면 받을 의미가 없다
제목 · 내용을 **공백 제거 후** 검사	템플릿대로(`min_length=1`)면 공백 한 칸짜리 문의가 저장된다. 닉네임(P1-08 37)과 같은 빈틈
제거는 앞뒤만	내용 안의 줄바꿈 · 문단은 문의의 일부 — 그대로 둔다 (시험으로 확인)
길이 상수를 화면과 같게 (`INQUIRY_TITLE_MAX` 새로)	화면 `maxlength` 200 · 2000 과 서버가 어긋나지 않게. 제목 길이는 템플릿에 숫자로만 있었다
이메일을 소문자로 바꾸지 않음	문의는 중복 확인 · 로그인에 쓰지 않는다. 사람이 적은 그대로 보관 (반복 전송 제한(51)이 비교할 때만 소문자로)
`privacy_agreed=false` 는 여기서 막지 않음	템플릿 설계 — 서비스(52)가 "Please agree to the privacy policy." 와 `details.privacy_agreed` 로 알려 줌 (가입과 같은 방식)

함께 생기는 변화: 52 `create_inquiry` 템플릿의 `data.title.strip()` 은 이제 할 필요가 없다 (이미 지운 값).

6. 시험 (`tests/test_inquiry_schemas.py` 새 파일 · 16개)

시험	확인한 것
정상	네 칸 그대로
공백 제거	제목 앞뒤 제거, 내용 앞뒤 제거 · 가운데 줄바꿈 유지, 이메일 앞뒤 제거
이메일 4가지	`@` 없음 · 점 없음 · `@` 둘 · 빈 글자 → `email` 오류
빈 제목 · 내용 6가지	빈 글자 · 공백만 · 줄바꿈만 × 두 칸 → 그 칸 오류
길이	200 · 2000 통과, 201 · 2001 오류, 앞뒤 공백 + 2000 통과
동의 칸 없음	오류

전체 시험 368개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 51 — `check_rate_limit()` · `record_inquiry_send()`

항목	내용
파일	`studionx/backend/app/services/inquiry_service.py`
작업	P1-12 문의 API — 반복 전송 제한 · PRD 01 FR-113
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	50 `InquiryCreateRequest` · 52 `create_inquiry()`

1. 하는 일

같은 사람이 문의를 짧은 시간에 계속 보내지 못하게 막는다. **같은 이메일 또는 같은 IP 로 10분 안에 2번** 저장되면, 3번째는 429.

상황 (같은 이메일 · 같은 IP)	결과
1번째 · 2번째	통과
3번째 (10분 안)	429 `TOO_MANY_REQUESTS` "Too many messages. Please try again later." · `details.retry_after` = 몇 초 뒤 가능한지
IP 는 다르지만 이메일이 같음 (`A@Example.com` 도 같은 이메일)	막힘
이메일은 다르지만 IP 가 같음	막힘
다른 IP · 다른 이메일	통과
저장에 **실패한** 시도 · 막힌 시도	**세지 않음**
첫 번째로부터 10분이 지남	한 자리가 비어 다시 1번 가능

2. 언제 쓰나

52 `create_inquiry` 안에서:

순서	함수
① 저장 전	`check_rate_limit(ip, email)` — 이미 2번이면 여기서 429 (저장하지 않음)
② 저장(commit) 성공 뒤	`record_inquiry_send(ip, email)` — 이번 것을 셈

사용자 결정 (2026-10-08)

정한 것	결정	템플릿
횟수 · 시간	**10분에 2번**	10분에 3번
저장에 실패한 시도도 세나	**세지 않음**	(정하기)

템플릿은 `check_rate_limit` 하나가 "검사하고 통과하면 바로 셈" 이었다. 그러면 검사를 통과한 뒤 **저장이 실패해도** 이미 세어진다. 결정대로 하려면 세는 일을 저장 **뒤로** 옮겨야 해서, 함수를 둘로 나눴다.

메모리 방식은 2026-10-03 결정 그대로 — 서버를 다시 켜면 기록이 비고, 서버가 여러 대면 따로 센다 (Phase 1 은 서버 한 대).

3. 코드

import (파일 맨 위)

```
import threading
import time
from collections import deque

from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Inquiry, User
from app.schemas.inquiry import InquiryCreateRequest
```

`threading` · `time` · `AppError` · `ErrorCode` 를 이번에 추가했다.

상수 · 기록 장소

```
INQUIRY_LIMIT_COUNT = 2  # 이 횟수까지 (2026-10-08 사용자 결정: 10분에 2번 — 템플릿은 3)
INQUIRY_LIMIT_SECONDS = 10 * 60  # 이 시간(초) 안에

_recent_sends: dict[str, deque[float]] = {}
_lock = threading.Lock()  # FastAPI 동기 함수는 여러 스레드에서 돎 → 기록을 함께 고치지 않게
_clock = time.monotonic  # 시스템 시계를 바꿔도 영향 없는 시각 (시험에서 바꿔 끼움)


def _keys(ip: str | None, email: str) -> list[str]:
    """제한을 셀 열쇠 — 이메일(소문자) 하나 + IP 가 있으면 IP 하나."""
    keys = [f"email:{email.strip().lower()}"]
    if ip:
        keys.append(f"ip:{ip}")
    return keys
```

함수

```
def check_rate_limit(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            q = _recent_sends.get(key)
            if q is None:
                continue
            while q and now - q[0] > INQUIRY_LIMIT_SECONDS:
                q.popleft()
            if not q:
                del _recent_sends[key]
                continue
            if len(q) >= INQUIRY_LIMIT_COUNT:
                raise AppError(
                    ErrorCode.TOO_MANY_REQUESTS,
                    429,
                    "Too many messages. Please try again later.",
                    details={"retry_after": int(INQUIRY_LIMIT_SECONDS - (now - q[0])) + 1},
                )
```

```
def record_inquiry_send(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            _recent_sends.setdefault(key, deque()).append(now)
```

문법 풀이

import 문법	뜻
`import threading`	여러 스레드가 같은 자료를 동시에 고치지 못하게 하는 잠금(`Lock`)
`import time`	`time.monotonic()` — 컴퓨터가 켜진 뒤 흐른 초. 시계를 바꿔도 뒤로 가지 않음
`from collections import deque`	양쪽 끝에 넣고 빼기가 빠른 목록 — 오래된 것은 앞에서 빼고(`popleft`) 새것은 뒤에 넣음(`append`)

식	뜻
`dict[str, deque[float]]`	열쇠(`"email:a@b.com"`)마다 저장 시각 목록
`with _lock:`	이 블록은 한 번에 한 스레드만 — 끝나면 저절로 풀림
`_clock = time.monotonic`	함수를 **변수에 담아 둠** — 시험에서 손으로 움직이는 시계로 바꿔 끼울 수 있게
`while q and now - q[0] > …: q.popleft()`	가장 오래된 기록부터 10분 넘은 것을 버림
`del _recent_sends[key]`	빈 목록은 열쇠째 지움 — 메모리가 계속 늘지 않게
`_recent_sends.setdefault(key, deque())`	없으면 빈 목록을 만들어 넣고, 있으면 그것을 돌려줌
`int(…) + 1`	남은 초를 올림 — 0초 남았다고 알려 주지 않게

4. 흐름

검사: 지금 시각 → 잠금 → 열쇠(이메일 · IP)마다 10분 넘은 기록 버리기 → 비면 지우기 → 2개 이상이면 429. **세기 (저장 성공 뒤):** 지금 시각 → 잠금 → 열쇠마다 시각 하나 추가.

5. 왜 이렇게

선택	이유
둘로 나눔 (검사 · 세기)	사용자 결정 "저장에 실패한 시도는 세지 않음" — 세는 시점을 저장 뒤로
이메일 **또는** IP	이메일만 보면 주소만 바꿔 계속 보낼 수 있고, IP 만 보면 같은 사람이 와이파이를 바꿔 보낼 수 있다. 둘 다 본다
이메일은 소문자 · 공백 제거로 비교	`A@Example.com` 과 `a@example.com` 은 같은 사람
IP 가 없으면 이메일만	시험 도구 등 IP 를 모르는 요청 (`ip:None` 이라는 열쇠를 만들지 않음 — 모두가 한 열쇠로 묶이는 것을 막음)
`time.monotonic`	서버 시계가 바뀌어도(시간 맞춤 등) 10분 계산이 틀어지지 않는다
잠금	FastAPI 의 동기 라우터는 스레드 여러 개에서 돈다. 동시에 같은 목록을 고치면 기록을 잃을 수 있다 (시험: 100개 스레드)

알아 둘 점 — 동시에 두 번: 같은 사람이 **같은 순간에** 두 번 보내면, 둘 다 검사를 통과한 뒤 저장되어 한도보다 1번 더 저장될 수 있다. 검사부터 저장까지를 한 잠금으로 묶으면 막을 수 있지만, 그동안 다른 사람의 문의까지

기다리게 된다. 문의 폼은 보내는 동안 버튼을 잠그므로(화면) 실제로는 드물다.

6. 시험 (`tests/test_inquiry_service.py` 새 파일 · 12개, 시계를 손으로 움직임)

시험	확인한 것
상수	2번 · 600초 (사용자 결정)
3번째	429 · 문구 · `retry_after` 601초
이메일 같고 IP 다름 (대소문자 · 공백도 다름) · IP 같고 이메일 다름	막힘
다른 사람	통과
IP 없음	이메일로만 셈, `ip:None` 열쇠 없음
검사만 5번 (저장 실패) 뒤 진짜 저장 2번	2번 모두 됨 → 3번째 막힘 — **실패는 세지 않음**
막힌 뒤 10분 지남	다시 됨 (막힌 것도 세지 않음)
`retry_after`	첫 기록 기준으로 줄어듦
10분 창이 밀림	첫 기록이 10분을 넘으면 1자리 남
오래된 기록	지워져 기록 장소가 빔
스레드 100개가 동시에 세기	하나도 잃지 않음

전체 시험 380개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 52 — `create_inquiry()`

항목	내용
파일	`studionx/backend/app/services/inquiry_service.py`
작업	P1-12 문의 API — 문의 저장 · PRD 01 FR-112 · FR-113
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	51 `check_rate_limit()` · 53 `POST /inquiries`

1. 하는 일

홈 화면 Contact 폼의 문의를 `inquiries` 표에 저장한다.

상황	결과
로그인하지 않은 사람	저장 — `user_id` 없음, 상태 `RECEIVED`, 동의 시각 = 지금
로그인한 회원	저장 — `user_id` 연결, 이메일은 **폼에 적은 답장 주소** 그대로
처리방침 동의 안 함	422 "Please agree to the privacy policy." · `details.privacy_agreed: "required"` — 저장 · 세기 없음
10분에 3번째	429 (51) — 저장 안 됨
저장 실패 (DB 오류)	오류 그대로 — 반복 전송으로 **세지 않음**

2. 언제 쓰나

쓰는 곳	
`POST /api/v1/inquiries` (53)	라우터가 로그인한 회원(없으면 `None`)과 IP 를 넘김
답변	관리자 화면(SQLAdmin `/admin`, P1-21)에서 `answer` · `answered_at` · 상태를 채움
보관 기간	답변 뒤 1년 지나면 지움 — 54 `purge_old_inquiries`

3. 코드

import (파일 맨 위)

```
import threading
import time
from collections import deque
from datetime import UTC, datetime

from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Inquiry, User
from app.schemas.inquiry import InquiryCreateRequest
```

`from datetime import UTC, datetime` 을 이번에 추가했다.

함수

```
def create_inquiry(
    db: Session,
    data: InquiryCreateRequest,
    user: User | None,
    ip: str | None,
) -> Inquiry:
    if not data.privacy_agreed:
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Please agree to the privacy policy.",
            details={"privacy_agreed": "required"},
        )
    check_rate_limit(ip, data.email)

    inquiry = Inquiry(
        user_id=user.user_id if user else None,
        email=data.email,
        title=data.title,
        content=data.content,
        privacy_agreed_at=datetime.now(UTC).replace(tzinfo=None),
    )
    db.add(inquiry)
    db.commit()
    db.refresh(inquiry)
    record_inquiry_send(ip, data.email)
    return inquiry
```

문법 풀이

식	뜻
`user: User | None`	로그인한 회원 또는 `None` (P1-03 06 `OptionalUser`)
`user.user_id if user else None`	조건식 — 회원이 있으면 번호, 없으면 `None`
`datetime.now(UTC).replace(tzinfo=None)`	UTC 지금 시각, 시간대 표시는 뗌 (DB 는 늘 UTC)
`db.refresh(inquiry)`	DB 가 채운 값(`inquiry_id` · `created_at` · `status`)을 다시 읽음

4. 흐름

1. 동의 확인 → 아니면 422.
2. 반복 전송 검사 (51) → 넘으면 429.
3. 문의 행을 만들어 저장.
4. 저장에 성공했으면 반복 전송 1번으로 셈.
5. 저장된 문의를 돌려줌.

5. 왜 이렇게

선택	이유
동의 검사가 맨 앞	동의 없이 온 요청은 반복 전송 검사도 하지 않는다 (세지도 않음)
세기는 `commit` **뒤**	사용자 결정 — 저장에 실패한 시도는 세지 않음 (51). `commit` 이 오류를 던지면 `record_inquiry_send` 줄까지 오지 않는다 (시험으로 확인)
로그인 회원도 **폼의 이메일**로 저장	회원이 다른 주소로 답장을 받고 싶을 수 있다. 회원 연결은 `user_id` 로 따로 남는다
`.strip()` 을 하지 않음 (템플릿에는 있음)	50 스키마가 이미 앞뒤 공백을 지운 값을 넘긴다
이름(`name`) 칸은 비워 둠	표에는 있지만 화면 폼에 이름 칸이 없다 (DB 는 `NULL` 허용)
상태는 DB 기본값	`RECEIVED` — 템플릿 · DB 설계 그대로
관리자 알림 메일 없음	Phase 1 범위 밖 (템플릿)

6. 시험 (`tests/test_inquiry_service.py` 에 5개 추가 · 모두 17개, 실제 MySQL)

시험	확인한 것
비회원	DB 에 `user_id` 없음 · 이메일 · 제목 · 내용(줄바꿈 포함) · `RECEIVED` · 동의 시각 · `created_at` · 답변 없음, 반복 전송 1번 셈
회원	`user_id` 연결, 이메일은 폼에 적은 다른 주소
동의 안 함	422 · 문구 · `details`, 저장 0 · 셈 0
3번째	429, 저장은 2개뿐
저장 실패 3번	오류 그대로, 셈 0 (사용자 결정)

전체 시험 385개 통과, `ruff` 검사 통과. 시험 뒤 DB 의 문의는 0개.

Backend(FastAPI/Python) 53 — `POST /api/v1/inquiries` (+ `client_ip()`)

항목	내용
파일	`studionx/backend/app/api/v1/inquiries.py` · `app/core/config.py` (설정 하나) · `studionx/.env.example` (한 줄)
작업	P1-12 문의 API — 문의 등록 라우터 · PRD 01 FR-112 · FR-113
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	52 `create_inquiry()` · 54 `purge_old_inquiries()`

1. 하는 일

홈 화면 Contact 폼의 문의를 받는다. **로그인은 선택** — 로그인했으면 회원과 연결한다.

요청	응답
`{ email, title: " Bug ", content, privacy_agreed: true }`	**201** `{ "inquiry_id": 12, "created_at": "…" }` — 제목은 `Bug` 로 저장
로그인 상태 (`Authorization: Bearer …`)	201, 회원 번호 연결
토큰이 틀림	201 — 비회원 문의로 받음
`privacy_agreed: false`	422 · `details.privacy_agreed`
이메일 `abc` · 제목 공백만 · 내용 없음	422 · 그 칸
10분에 3번째	**429** `TOO_MANY_REQUESTS` "Too many messages. Please try again later." · `details.retry_after`

2. 언제 쓰나

쓰는 곳	
화면	home.html Contact 폼 `initContactForm` (P1-11 남은 JS — 이 작업 뒤 연결). 201 → 폼 비우고 안내, 429 → 문구
이어지는 것	50 스키마 → 52 `create_inquiry` (동의 → 51 반복 전송 검사 → 저장 → 셈)

사용자 결정 (2026-10-08) — 보낸 사람 IP 를 어디서 읽나 (결정 A)

상황	읽는 곳
기본 (`TRUST_PROXY=false`)	연결된 상대의 IP `request.client.host` — **`X-Forwarded-For` 헤더는 무시**
배포 때 프록시(Nginx 등)를 두면 `TRUST_PROXY=true`	`X-Forwarded-For` 의 **첫** IP (원래 보낸 사람)

프록시 뒤에서는 연결 상대가 늘 프록시라 모든 사람이 한 IP 로 세어진다. 그래서 헤더가 필요하지만, 프록시가 **없을** 때 헤더를 믿으면 누구나 헤더를 꾸며 IP 제한을 피할 수 있다. 그래서 기본은 끄고, 배포 때 설정 하나로 켠다. (이메일 기준 제한은 어느 쪽이든 동작)

바꾼 설정 파일	내용
`app/core/config.py`	`trust_proxy: bool = False` + 설명 주석
`studionx/.env.example`	`TRUST_PROXY=false` + 설명 (실제 `.env` 는 건드리지 않음)

3. 코드

import (파일 맨 위)

```
from fastapi import APIRouter, Request, status

from app.api.deps import Db, OptionalUser
from app.core.config import get_settings
from app.schemas.common import ErrorResponse
from app.schemas.inquiry import InquiryCreateRequest, InquiryCreateResponse
from app.services import inquiry_service
```

`get_settings` · `inquiry_service` 줄을 이번에 추가했다.

도우미

```
def client_ip(request: Request) -> str | None:
    if get_settings().trust_proxy:
        first = request.headers.get("x-forwarded-for", "").split(",")[0].strip()
        if first:
            return first
    return request.client.host if request.client else None
```

라우터 (경로 · 응답 설정은 템플릿 그대로)

```
@router.post(
    "",
    response_model=InquiryCreateResponse,
    status_code=status.HTTP_201_CREATED,
    responses={422: {"model": ErrorResponse}, 429: {"model": ErrorResponse}},
)
def create_inquiry(
    data: InquiryCreateRequest,
    request: Request,
    user: OptionalUser,
    db: Db,
) -> InquiryCreateResponse:
    inquiry = inquiry_service.create_inquiry(db, data, user, client_ip(request))
    return InquiryCreateResponse(inquiry_id=inquiry.inquiry_id, created_at=inquiry.created_at)
```

문법 풀이

import 문법	뜻
`from app.core.config import get_settings`	`.env` 설정 (P1-01). `trust_proxy` 를 읽음
`from app.services import inquiry_service`	모듈 통째로 — 라우터 함수 이름도 `create_inquiry` 라 겹치지 않게

식	뜻
`request.headers.get("x-forwarded-for", "")`	헤더 이름은 대소문자를 가리지 않음, 없으면 빈 글자
`.split(",")[0].strip()`	`"203.0.113.5, 10.0.0.1"` → `"203.0.113.5"` (맨 앞이 원래 보낸 사람, 뒤는 거쳐 온 프록시들)
`request.client.host if request.client else None`	연결 상대 IP. 알 수 없으면 `None` (51 은 그때 이메일로만 셈)
`user: OptionalUser`	토큰이 있고 맞으면 회원, 없거나 틀리면 `None` (P1-03 06) — 401 을 내지 않음

4. 흐름

1. 본문 검사 (50) — 틀리면 422.
2. 토큰이 있으면 회원 확인 (없거나 틀리면 비회원).
3. 보낸 사람 IP 결정 (`client_ip`).
4. 52 `create_inquiry` — 동의 422 · 반복 전송 429 · 저장.
5. 201 + 번호 · 접수 시각.

5. 왜 이렇게

선택	이유
로그인 선택 · 틀린 토큰도 받음	문의는 누구나 보낸다. 만료된 토큰 때문에 문의가 막히면 안 된다
응답은 번호 · 시각만	템플릿 — 화면은 "보냈습니다" 만 보여 준다. 받은 내용을 그대로 돌려줄 필요가 없다
헤더는 **첫** IP	프록시들은 자기 앞의 주소를 뒤에 덧붙인다 — 맨 앞이 사용자
설정은 `.env` 로	개발 컴퓨터(프록시 없음)와 배포 서버(프록시 있음)가 코드를 바꾸지 않고 다르게 동작

6. 시험 (`tests/test_inquiry_routes.py` 에 15개 추가 · 모두 16개, 실제 앱 + MySQL)

시험	확인한 것
비회원	201, 응답 칸 2개, DB 에 저장 · 제목 공백 제거 · `user_id` 없음
회원 · 틀린 토큰	회원 연결 · 비회원으로 201
동의 안 함 · 형식 3가지	422 · `details` 에 그 칸
3번째	429 · 코드 · 문구 · `retry_after` 1~601초
기본: 헤더를 꾸며도	IP 마다 다른 `X-Forwarded-For` · 다른 이메일로 보내도 3번째 429 — 헤더를 믿지 않음
`TRUST_PROXY` 켬	헤더의 첫 IP 로 사람마다 따로 셈 (3명 모두 201), 연결 IP 는 쓰지 않음
`client_ip` 5가지	꺼짐 → 연결 IP / 켜짐 → 첫 IP · 공백 제거 / 빈 헤더 · 헤더 없음 → 연결 IP

전체 시험 400개 통과, `ruff` 검사 통과. 시험 뒤 DB 의 문의는 0개.

Backend(FastAPI/Python) 54 — `purge_old_inquiries()`

항목	내용
파일	`studionx/backend/app/workers/retention.py`
작업	P1-12 문의 API — 오래된 문의 지우기 · 개인정보 처리방침 §4
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	53 `POST /inquiries` · 55 `retention.main()`

1. 하는 일

하루 한 번 도는 정리 작업 중 **문의** 쪽. 오래된 문의를 DB 에서 아예 지운다.

문의 (오늘 기준)	결과
답변한 지 400일	지움
받은 지 500일, 답변한 지 300일	**남김** — 답변 날부터 1년
답변 없이 받은 지 400일	지움 (사용자 결정)
답변 없이 받은 지 100일	남김
답변한 지 정확히 365일	지움

돌려주는 값 = 지운 개수.

2. 언제 쓰나

실행	
55 `main()`	`python -m app.workers.retention` — 하루 한 번 (예약 작업은 배포 때). 49 `purge_withdrawn_accounts` 와 함께

사용자 결정 (2026-10-08) — 답변하지 않은 문의

처리방침에는 "답변 후 1년" 만 있어, 관리자가 답하지 않은 문의는 영영 남을 수 있었다. **받은 뒤 1년**에 지우기로 했다 (추천 A).

문의	기준 날짜	기간
답변함	답변한 날	1년
답변 안 함	받은 날	1년

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import delete, func, or_, select
```

```
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models import Inquiry, User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`delete` · `func` · `Inquiry` 를 이번에 추가했다.

상수

```
INQUIRY_KEEP_DAYS = 365  # 답변 후 · (답변 없으면) 받은 뒤 보관 기간 (처리방침 §4, 2026-10-08)
```

함수

```
def purge_old_inquiries(db: Session, now: datetime) -> int:
    if now.tzinfo is not None:
        now = now.astimezone(UTC).replace(tzinfo=None)
    cutoff = now - timedelta(days=INQUIRY_KEEP_DAYS)
    since = func.coalesce(Inquiry.answered_at, Inquiry.created_at)
    count = db.execute(delete(Inquiry).where(since <= cutoff)).rowcount
    db.commit()
    return count
```

문법 풀이

import 문법	뜻
`from sqlalchemy import delete, func`	`DELETE` 문 · SQL 함수 부르기 (`func.coalesce` → SQL `COALESCE`)

식	뜻
`func.coalesce(Inquiry.answered_at, Inquiry.created_at)`	SQL `COALESCE(answered_at, created_at)` — 답변 시각이 있으면 그것, 없으면 (`NULL`) 받은 시각. 두 규칙을 **조건 하나**로
`delete(Inquiry).where(…)`	조건에 맞는 행을 DB 에서 한 번에 지움 (하나씩 읽지 않음)
`.rowcount`	지운 행 수

4. 흐름

1. 기준 시각 = 지금(UTC) − 365일.
2. "답변 시각, 없으면 받은 시각" 이 기준보다 오래된 문의를 한 번에 지운다.
3. 저장하고 지운 수를 돌려준다.

5. 왜 이렇게

선택	이유
받은 뒤 1년 (답변 없을 때)	사용자 결정 — 답장 이메일 같은 개인정보가 끝없이 남지 않게 (저장 기간 제한 원칙)
답변이 있으면 **답변 날** 기준	처리방침 그대로 — 늦게 답한 문의도 답한 뒤 1년은 남긴다 (시험: 받은 지 500일 · 답변 300일 → 남김)
`COALESCE` 한 조건	두 규칙을 따로 지우는 것보다 단순하고, 한 번의 `DELETE` 로 끝난다

선택	이유
행을 진짜로 지움 (탈퇴 회원처럼 비우지 않음)	문의는 다른 표가 가리키지 않는다 (템플릿)
49 와 같은 시간대 처리	한국 시각을 넘겨도 정확히 1년

6. 처리방침 문구

이 결정에 맞춰 처리방침에 "답변하지 않은 문의는 받은 뒤 1년" 을 넣고 버전을 `2026-10-08` 로 올렸다 (사용자 결정 A) — 55 §6.

7. 시험 (`tests/test_retention.py` 에 3개 추가 · 모두 11개, 실제 MySQL)

시험	확인한 것
5가지 문의	3개 지움 — 답변 400일 · 답변 없이 400일 · 답변 정확히 365일 / 남음 — 답변 300일(받은 지 500일) · 답변 없이 100일
두 번 돌리기	1 → 0
한국 시각 21:00 (= UTC 12:00)	정확히 1년 된 문의 지움

전체 시험 403개 통과, `ruff` 검사 통과. 시험 뒤 DB 의 문의는 0개.

Backend(FastAPI/Python) 55 — `retention.main()` (+ 처리방침 버전 2026-10-08)

항목	내용
파일	`studionx/backend/app/workers/retention.py` / 처리방침: `privacy.html` · `home.html` · `join.js` · `schemas/auth.py` · `docs/Privacy_Policy_Draft.md`
작업	P1-12 문의 API — 보관 기간 정리를 하루 한 번 실행하는 명령 · 처리방침 §4
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	54 `purge_old_inquiries()` · (P1-12 마지막)

1. 하는 일

명령 한 줄로 **보관 기간 정리 두 가지**를 차례로 돌리고 결과를 한 줄 출력한다.

```
cd studionx/backend
..\.venv\Scripts\python -m app.workers.retention
# → withdrawn accounts: 0, inquiries: 0
```

차례	정리	함수
①	탈퇴 3일 지난 계정의 개인 정보 비우기	49 `purge_withdrawn_accounts` (P1-09)
②	답변 1년 · (답변 없으면) 받은 뒤 1년 지난 문의 지우기	54 `purge_old_inquiries`

정리 중 오류가 나면 예외를 그대로 올린다 → Sentry 로 보고되고, 명령은 실패로 끝난다(예약 작업이 실패를 알 수 있게).

2. 언제 쓰나

하루 한 번 운영체제의 예약 작업으로 돌린다 — Windows 작업 스케줄러 또는 리눅스 cron. 예약은 **배포할 때** 정한다 (템플릿 머리말). 지금은 손으로 한 번 실행해 동작을 확인했다 (실제 DB · Sentry 끔 → `withdrawn accounts: 0, inquiries: 0`, 종료 코드 0 — 지울 것이 없는 상태).

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import delete, func, or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.sentry import init_sentry
from app.db.session import SessionLocal
from app.models import Inquiry, User
```

```
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`init_sentry` · `SessionLocal` 줄을 이번에 추가했다.

함수

```
def main() -> None:
    init_sentry(get_settings())
    now = datetime.now(UTC)
    with SessionLocal() as db:
        accounts = purge_withdrawn_accounts(db, now)
        inquiries = purge_old_inquiries(db, now)
    print(f"withdrawn accounts: {accounts}, inquiries: {inquiries}")
```

파일 맨 아래의 `if __name__ == "__main__": main()` 은 템플릿 그대로 — `python -m app.workers.retention` 으로 실행하면 `main()` 이 불린다.

문법 풀이

import 문법	뜻
`from app.core.sentry import init_sentry`	서버(P1-01)와 같은 Sentry 설정 — 명령으로 따로 돌 때도 오류를 보고하려고
`from app.db.session import SessionLocal`	DB 세션 공장 — 서버 요청 밖에서 직접 세션을 만듦

식	뜻
`with SessionLocal() as db:`	세션을 열고, 블록이 끝나면(오류가 나도) 닫음
`datetime.now(UTC)`	시간대가 붙은 지금 시각 — 49 · 54 가 UTC 로 바꿔 비교
`f"…{accounts}…"`	글자 안에 값 끼우기
`python -m app.workers.retention`	파일을 **모듈**로 실행 — `app` 패키지를 찾을 수 있게 `studionx/backend` 에서

4. 흐름

1. Sentry 켬.
2. 지금 시각을 한 번 잼.
3. 세션을 열고 탈퇴 계정 정리 → 문의 정리 (각자 저장).
4. 결과 한 줄 출력. 오류면 그대로 올라감.

5. 왜 이렇게

선택	이유
기준 시각을 한 번만	두 정리가 같은 "지금" 을 기준으로 — 기록을 읽을 때 헷갈리지 않게
오류를 삼키지 않음	정리가 조용히 실패하면 처리방침의 약속(삭제)을 어기게 된다. Sentry 보고 + 실패 종료 코드로 알 수 있게
Sentry 를 먼저	그 뒤의 어떤 오류든 보고되게
한 줄 출력	예약 작업 기록에서 날마다 몇 개를 지웠는지 볼 수 있다

6. 처리방침 버전 올림 (사용자 결정 A, 2026-10-08)

54 의 결정("답변하지 않은 문의는 받은 뒤 1년")을 처리방침에 넣고, privacy.html 의 규칙("내용을 바꾸면 버전도 올림")대로 버전을 `2026-10-05` → **`2026-10-08`** 로 올렸다. 아직 공개 전(Draft)이라 다시 동의를 받을 사람은 없고, 앞으로 가입하는 사람은 새 버전으로 기록된다.

파일	바꾼 것
`privacy.html` §4	"Messages you send us — 1 year after we reply. **If we never reply, 1 year after you send them.**" · Version 줄
`home.html` 문의 요약	"One year after we reply — **or one year after you send it, if we never reply** — then we delete it." · Version 줄
`join.js` · `schemas/auth.py`	`PRIVACY_VERSION = '2026-10-08'` (가입 때 보내고 서버가 확인하는 값 — 둘이 같아야 가입됨)
`docs/Privacy_Policy_Draft.md`	상태 · §2.4 보관 표 · 버전 줄 + **버전 기록 표** (10-05 첫 확정판 / 10-08 문의 보관) + PDF
`tests/test_auth_service.py`	가입 시험 본문의 `"2026-10-05"` 를 상수 `PRIVACY_VERSION` 으로 (버전이 바뀌어도 시험이 따라감)

7. 시험 (`tests/test_retention.py` 에 2개 추가 · 모두 13개)

시험	확인한 것
`main()` (세션 · Sentry 를 시험용으로 바꿔 끼움)	Sentry 를 먼저 켬, 탈퇴 10일 계정 · 400일 문의를 정리, 출력 `withdrawn accounts: N, inquiries: M`
정리 중 오류	예외가 그대로 올라옴
(손으로) 실제 명령	`withdrawn accounts: 0, inquiries: 0`, 종료 코드 0

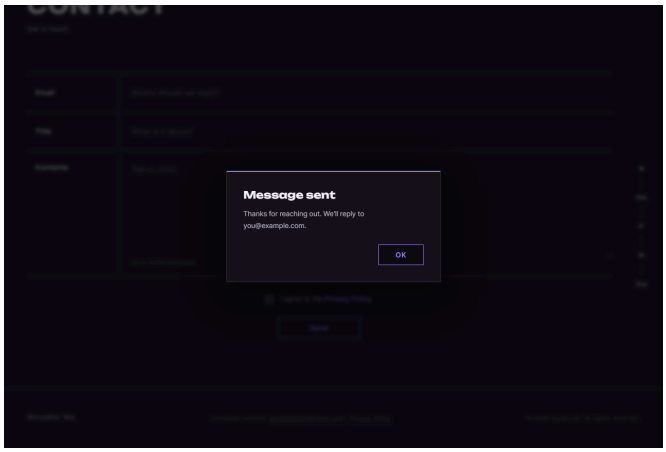
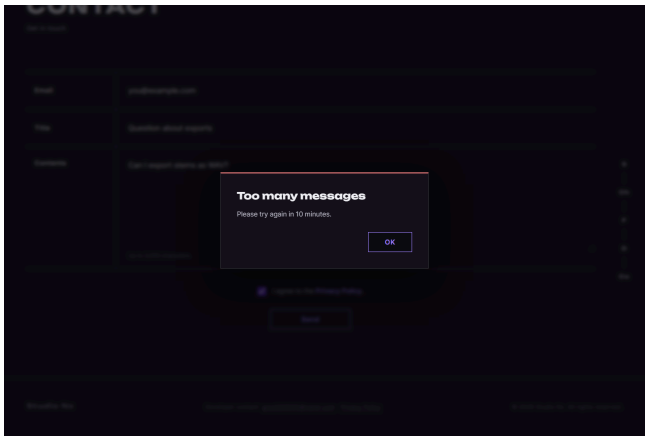
전체 시험 405개 통과 (처리방침 버전을 올린 뒤에도), `ruff` 검사 통과.

JS 28 — `initContactForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`
작업	P1-12 문의 — 홈 Contact 폼 보내기 · 로그인 시 이메일 자동 채움 · PRD 01 FR-112 · FR-113 (P1-11 화면의 남은 JS)
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	27 `initLogoutButton()` · (P1-12 마지막)

1. 하는 일

홈 화면 **Contact** 폼의 **Send** 를 서버(백엔드 53)에 연결하고, 결과를 알림창으로 알려 준다.

보냄	이미 3번째 (10분에 2번 넘음)
	

상황	화면
로그인한 사람이 홈을 열면	이메일 칸에 **내 이메일이 미리 채워짐** (비어 있을 때만)
빈 칸 · 형식 · 동의 안 함	보내지 않고 알림창 (FIX-01 19 — 브라우저 검사)
보내는 중	Send 잠금
성공	알림 "Message sent — Thanks for reaching out. We'll reply to you@example.com." · 폼 비움 (내 이메일은 다시 채움)
10분에 3번째 (서버 429)	빨간 알림 "Too many messages — Please try again in 10 minutes." · 폼은 그대로
그 밖의 서버 오류	빨간 알림 "Couldn't send your message" + 서버 문구 · 폼 그대로

2. 언제 쓰나

home.html 이 열릴 때 한 번. 이어지는 것들:

순서	하는 곳
① 칸 검사 · 알림	브라우저 기본 검사 + FIX-01 19 `initFormAlerts` — 통과해야 `submit` 이 일어남

순서	하는 곳
② 보내기	백엔드 53 `POST /inquiries` → 50 · 51 · 52
③ 결과	FIX-01 18 `showAlert`
(자동 채움)	P1-02 `isLoggedIn` → P1-08 `GET /users/me`

결과를 알림창으로 보여 주므로 **HTML 은 바꾸지 않았다** (폼에 결과 문구 자리가 없었다).

3. 코드

import (`home.js` 맨 위)

```
import { initAuthState, initLoginForm, initPasswordToggles, initOAuthPlaceholders } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { parseLoginHash, isLoggedIn } from '../core/auth.js';
import { initFormAlerts, showAlert } from '../core/dialog.js';
import { api } from '../core/api.js';
```

`isLoggedIn` · `showAlert` · `api` 를 이번에 추가했다.

함수

```
export function initContactForm() {
  const form = document.getElementById('contact-form');
  const email = document.getElementById('contact-email');
  const title = document.getElementById('contact-subject');
  const content = document.getElementById('contact-content');
  const agree = document.getElementById('contact-agree');
  if (!form || !email || !title || !content || !agree) return;
  const submit = form.querySelector('button[type="submit"]');
  let myEmail = '';

  // 로그인 상태면 내 이메일을 미리 채움 (기다리지 않음 — 폼은 바로 쓸 수 있음)
  isLoggedIn().then(async (loggedIn) => {
    if (!loggedIn) return;
    try {
      myEmail = (await api.get('/users/me')).email;
    } catch {
      return; // 못 받아도 문의는 직접 적어 보낼 수 있음
    }
    if (email.value === '') email.value = myEmail;
  });

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    if (submit) submit.disabled = true;
    try {
      const sentTo = email.value.trim();
      // 토큰을 붙여 보냄 → 로그인한 회원이면 서버가 문의를 회원과 연결 (FR-112).
      // 토큰이 없거나 틀려도 이 API 는 401 이 아니라 비회원으로 받음 (백엔드 53)
      await api.post('/inquiries', {
        email: sentTo,
```

```
        title: title.value,
        content: content.value,
        privacy_agreed: agree.checked,
      });
      form.reset();
      if (myEmail) email.value = myEmail;
      showAlert('Message sent', `Thanks for reaching out. We'll reply to ${sentTo}.`); // 닫힐 때까지 기다리지 않음 — 버튼은 바로 다시 켬
    } catch (err) {
      if (err.status === 429) {
        const wait = err.details && err.details.retry_after ? err.details.retry_after : 600;
        const minutes = Math.ceil(wait / 60);
        showAlert(
          'Too many messages',
          `Please try again in ${minutes} minute${minutes === 1 ? '' : 's'}.`,
          { tone: 'error' },
        );
      } else {
        showAlert("Couldn't send your message", err.message, { tone: 'error' });
      }
    } finally {
      if (submit) submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const wait = err.details?.retry_after ?? 600;
…
const { email: mine } = await api.get('/users/me');
email.value ||= mine;
```

최신 문법	실제 코드	뜻
`err.details?.retry_after ?? 600`	`err.details && err.details.retry_after ? err.details.retry_after : 600`	있으면 그 값, 없으면 600
`const { email: mine } = …`	`myEmail = (await …).email`	응답에서 칸 꺼내며 이름 바꾸기
`a ||= b`	`if (a === '') a = b`	비어 있을 때만 넣기

문법 풀이

import 문법	뜻
`import { …, isLoggedIn } from '../core/auth.js'`	P1-02 06 — 토큰이 없으면 재발급을 시도해 로그인 여부를 알려 줌
`import { …, showAlert } from '../core/dialog.js'`	FIX-01 18 — 알림창

식	뜻
`isLoggedIn().then(async (loggedIn) => { … })`	**기다리지 않고** 결과가 오면 실행 — 그동안에도 폼은 쓸 수 있음
`catch { … }` (괄호 없음)	오류 내용은 쓰지 않을 때 이렇게 줄여 씀 (`api.js` 도 같은 방식)
`form.reset()`	모든 칸을 처음 값으로 (체크박스도 해제)
`Math.ceil(wait / 60)`	초 → 분, 올림 (예: 599초 → 10분)

식	뜻
`` `minute${minutes === 1 ? '' : 's'}` ``	1이면 minute, 아니면 minutes
`showAlert(…)` (`await` 없음)	알림창이 닫히기를 **기다리지 않음** — 아래 §5

4. 흐름

열 때: 로그인 확인 → 로그인이면 내 이메일을 받아, 칸이 비어 있으면 채움. **보낼 때:** (브라우저 검사 통과) → Send 잠금 → 보냄 → 성공이면 폼 비우고(내 이메일 다시) 알림, 429 면 "몇 분 뒤", 그 밖은 서버 문구 → Send 다시 켬.

5. 왜 이렇게

선택	이유
결과를 알림창으로	폼에 결과 문구 자리가 없고, FIX-01 에서 모든 알림을 같은 모양으로 쓰기로 했다
알림창을 **기다리지 않음**	시험에서 찾은 문제: 처음엔 `await showAlert(…)` 였는데, 그러면 사용자가 OK 를 누를 때까지 `finally`(Send 다시 켜기)가 실행되지 않았다
막히거나 실패하면 폼을 비우지 않음	적은 문의가 사라지면 다시 써야 한다
429 는 "몇 분 뒤" 로	서버가 `retry_after`(초)를 준다 (51) — 사람은 분으로 읽는다
토큰을 붙여 보냄 (`api.post` 기본)	로그인한 회원이면 서버(52)가 문의에 `user_id` 를 연결한다 (FR-112). 이 API 는 토큰이 없거나 틀려도 401 이 아니라 비회원으로 받으므로(53) 문의가 막히지 않는다
자동 채움은 칸이 **비어 있을 때만**	사용자가 먼저 다른 주소를 적었으면 덮어쓰지 않는다
자동 채움을 기다리지 않음	로그인 확인 · 내 정보 요청 동안에도 폼을 바로 쓸 수 있게

처음 만든 것과 바꾼 점 — 회원 연결: 처음에는 `{ auth: false }`(토큰 없이)로 보냈는데, 그러면 로그인한 회원의 문의도 **회원과 연결되지 않았다** (FR-112 빈틈). 이 API 는 토큰 문제로 401 을 내지 않으므로 토큰을 붙여 보내도록 고쳤고, DB 에서 회원 문의는 `user_id` 가 연결되고 비회원 문의는 비어 있음을 확인했다.

6. 시험 (실제 home.html · 실제 서버 · 9개)

반복 전송 제한은 서버 메모리에 세므로, 시험마다 시험 서버를 다시 켜 기록을 비웠다.

시험	확인한 것
비회원	이메일 칸 빔
보내기	보내는 동안 Send 잠금 → "Message sent" · 받는 주소 → 폼 비움 · Send 다시 켬
3번째	빨간 "Too many messages" · "Please try again in 10 minutes." · 폼 그대로
서버 오류 (가짜 500)	"Couldn't send your message" · 서버 문구
로그인한 사람	이메일이 미리 채워짐 → 보낸 뒤 폼을 비워도 내 이메일은 다시 채움 → **DB 에서 그 문의의 `user_id` = 그 회원** (비회원 문의는 비어 있음)

처음 6/7 — §5 의 "알림을 기다려 Send 가 잠긴 채" 를 찾아 고친 뒤 7/7, 로그인 시험 2/2. 회원 연결 빈틈(위)도 고친 뒤 다시 돌려 통과. 홈 화면의 앞 시험(P1-07 13 · 14, FIX-01 18 · 19)도 다시 돌려 모두 통과. 시험이 만든 문

의 · 계정은 지움.